

Lab-03 Report: Generated 2026-09-28 22:22

Names:

Group Name:

- **Q1.1:** Brainstorm ways that classification could be used in your field (Electrical, Computer or Cybersecurity Engineering). How could it be used to make your work more efficient and effective?
- **A1.1:** Classification would be useful for quickly filtering or determining a particular input, such as object detection on a self driving car. It would be almost necessary for real-world work in which you are dealing with varied scenarios that you cannot realistically account for.
- **Q2.1:** Map the string labels to the values 0 and 1. This can be done in 1 line. After printing these binary labels, what have you learned about the TUANDROMD dataset, and how might that impact how you load the data for training?
- **A2.1:** After accidentally converting them all into 1's and then attempting to use map() I succeeded in reclassifying. After that and rerunning the describe cell I noticed that the average is around 0.8 which means there is likely an inherent bias towards malware in this dataset.

```
# Code for Q2.1 here. Convert the "Label" column to binary values (0 for
goodware, 1 for malware) and print the resulting column.
# YOUR CODE STARTS HERE
df1['Label'] = df1['Label'].map({'goodware': 0, 'malware': 1})
# YOUR CODE ENDS HERE

print(df1['Label'])
```

```

0      1
1      1
2      1
3      1
4      1
..
4459   0
4460   0
4461   0
4462   0
4463   0
Name: Label, Length: 4464, dtype: int64

```

- **Q2.2:** Update the code block below to plot a histogram to visualize how the two classes are balanced. Briefly reflect on the class distribution you see, and how you anticipate it could effect training.
- **A2.2:** The split of malware to goodware is skewed heavily towards malware.

```

# Use plt.hist to display a histogram of the "Label" column, showing the
# distribution of goodware and malware samples.
# Set the number of bins to 2

# YOUR CODE STARTS HERE
# plt.hist(df1['Label'].to_numpy(), bins=2) original, then used genAI to spruce
# it up

# YOUR CODE STARTS HERE
# Plot histogram with 2 bins, bar spacing, and borders
counts, bins, patches = plt.hist(
    df1['Label'].to_numpy(),
    bins=2,
    rwidth=0.8,
    color='steelblue',
    edgecolor='black'
)

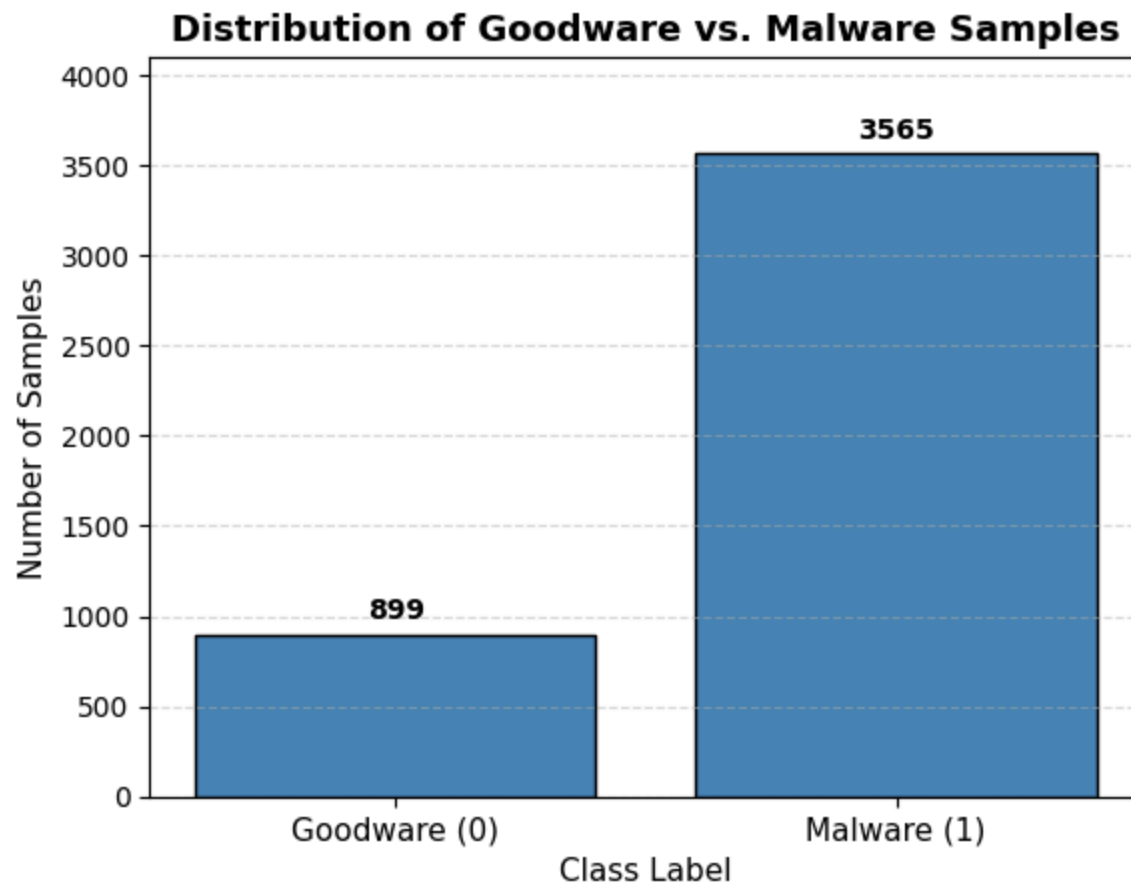
# Align tick labels with the center of each bin (0.25 and 0.75)
bin_centers = 0.5 * (bins[:-1] + bins[1:])
plt.xticks(bin_centers, ['Goodware (0)', 'Malware (1)'], fontsize=11)

# Annotate sample counts on top of each bar
for count, patch in zip(counts, patches):
    plt.text(
        patch.get_x() + patch.get_width() / 2,
        count + 50,
        f'{int(count)}',
        ha='center',
        va='bottom',
        fontweight='bold'
    )

```

```
# Formatting
plt.title('Distribution of Goodware vs. Malware Samples', fontsize=13,
fontweight='bold')
plt.xlabel('Class Label', fontsize=11)
plt.ylabel('Number of Samples', fontsize=11)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.ylim(0, max(counts) * 1.15) # Leave headroom for labels
# YOUR CODE ENDS HERE
# YOUR CODE ENDS HERE
```

(0.0, 4099.75)



- **Q2.3:** Update the `df1` DataFrame by excluding the label column and converting all remaining values to numeric. If a value cannot be converted, replace it with NaN. One option is to use the Pandas `to_numeric` to convert arguments to numeric types, with the option `errors='coerce'` to force NaN. Briefly describe how your code works below.
- **A2.3:** From `df1`, it drops the column "Label" and then it stores the other features into `feature_cols`. `pd.to_numeric` converts all of the values of the features into a numeric value. A NaN implies it cannot be converted into a numeric value. The `describe()` function prints them out into the notebook.

```
# Code for Q2.3 here. Store a subset of df1 as "feature_cols" and convert all
data to numeric, replacing with "NaN" if impossible.
# YOUR CODE STARTS HERE
feature_cols = df1.drop(columns='Label').columns
print(feature_cols)
df1[feature_cols] = df1[feature_cols].apply(pd.to_numeric, errors='coerce')
df1.describe()
# YOUR CODE ENDS HERE
```

```
Index(['ACCESS_ALL_DOWNLOADS', 'ACCESS_CACHE_FILESYSTEM',
      'ACCESS_CHECKIN_PROPERTIES', 'ACCESS_COARSE_LOCATION',
      'ACCESS_COARSE_UPDATES', 'ACCESS_FINE_LOCATION',
      'ACCESS_LOCATION_EXTRA_COMMANDS', 'ACCESS_MOCK_LOCATION',
      'ACCESS_MTK_MMHW', 'ACCESS_NETWORK_STATE',
      ...,
      'Landroid/content/pm/PackageManager;->getInstalledPackages',
      'Landroid/telephony/TelephonyManager;->getLine1Number',
      'Landroid/telephony/TelephonyManager;->getNetworkOperator',
      'Landroid/telephony/TelephonyManager;->getNetworkOperatorName',
      'Landroid/telephony/TelephonyManager;->getNetworkCountryIso',
      'Landroid/telephony/TelephonyManager;->getSimOperator',
      'Landroid/telephony/TelephonyManager;->getSimOperatorName',
      'Landroid/telephony/TelephonyManager;->getSimCountryIso',
      'Landroid/telephony/TelephonyManager;->getSimSerialNumber',
      'Lorg/apache/http/impl/client/DefaultHttpClient;->execute'],
      dtype='str', length=241)
```

	ACCESS_ALL_DOWNLOADS	ACCESS_CACHE_FILESYSTEM	ACCESS_CHECKIN_F
count	4464.000000	4464.000000	,
mean	0.000896	0.000896	
std	0.029924	0.029924	
min	0.000000	0.000000	
25%	0.000000	0.000000	
50%	0.000000	0.000000	
75%	0.000000	0.000000	
max	1.000000	1.000000	

8 rows × 242 columns

- **Q2.4:** To better understand the dataset, briefly research `WAKE_LOCK`, the most common feature. What does it represent, and why might it appear so frequently in Android applications?

- **A2.4:** WAKE_LOCK is a feature that allows an app to keep using the device's processor or screen when it would otherwise go to sleep. I would assume it would show up frequently to do background tasks, both malicious and benign.
- **Q2.5:** Determine the most frequently occurring malware sample feature, and briefly research it. What does this permission allow, and why might it be strongly associated with malicious applications?
- **A2.5:** RECEIVE_BOOT_COMPLETED, this allows an app to run code immediately upon booting up. It's likely that malicious applications, such as a keylogger, would need to run in the background at all times including immediately after boot.
- **Q3.1:** Finish the `__init__` implementation below, creating `X`, `y`, and `num_train`. Don't forget to save them as attributes of the class using `self`. You will likely need to use `torch.unsqueeze()` to `self.y` to make it compatible with our loss function to come - explain why this is the case.
- **A3.1:**

```
class MalwareDataset(d2l.DataModule):
    def __init__(self, malware_data_df, train_val_split=0.8, batch_size=32):
        super().__init__()
        self.save_hyperparameters()
        #Create a copy - this allows for reruns without effecting notebook's
global state
        df = malware_data_df.copy()

        # Convert the labels in the "Label" column to binary values (0 for
goodware, 1 for malware)
        df["Label"] = df["Label"].map({"goodware": 0, "malware": 1})

        #Grab the top 30 features
        feature_cols = df.columns.difference(['Label'])
        feature_freq = df[feature_cols].sum().sort_values(ascending=False)
        top_features = feature_freq.head(30).index

        #List Position preserves the natural order of the columns
        final_cols = list(top_features) + ['Label']
        df = df[final_cols]

        #Shuffle Entries
        df = df.sample(frac=1).reset_index(drop=True)

        # YOUR CODE STARTS HERE
```

```

        self.X = torch.tensor(df[top_features].to_numpy(),
dtype=torch.float32)
        self.y = torch.unsqueeze(torch.tensor(df['Label'],
dtype=torch.float32), dim=1)

        self.num_train = int(len(df) * train_val_split)
        self.num_val = len(df) - self.num_train
        # YOUR CODE ENDS HERE

@d2l.add_to_class(MalwareDataset) #@save
def get_dataloader(self, train):
    i = slice(0, self.num_train) if train else slice(self.num_train, None)
    return self.get_tensorloader((self.X, self.y), train, i)

```

- **Q3.2):** Plot a histogram of the output feature (`y`) returned after calling `.train_dataloader()`. Compare to the previously reviewed values of the corresponding feature from the initial exploratory data analysis. Should we have expected the bars to be of equal height? Why or why not?
- **A3.2):** The proportions of goodware compared to the number of malware is about the same. No, we shouldn't have expected equal height as we selected at random, so we would expect a similar proportion as the whole dataset, which is unbalanced.

```

# Create a MalwareDataset object using the original dataset
malware_df= pd.read_csv("data/TUANDROMD.csv")
MalwareData = MalwareDataset(malware_df)

# Generate first iteration of a dataloader
X, y = next(iter(MalwareData.train_dataloader()))

# Plot this histogram of the output feature in the first training batch

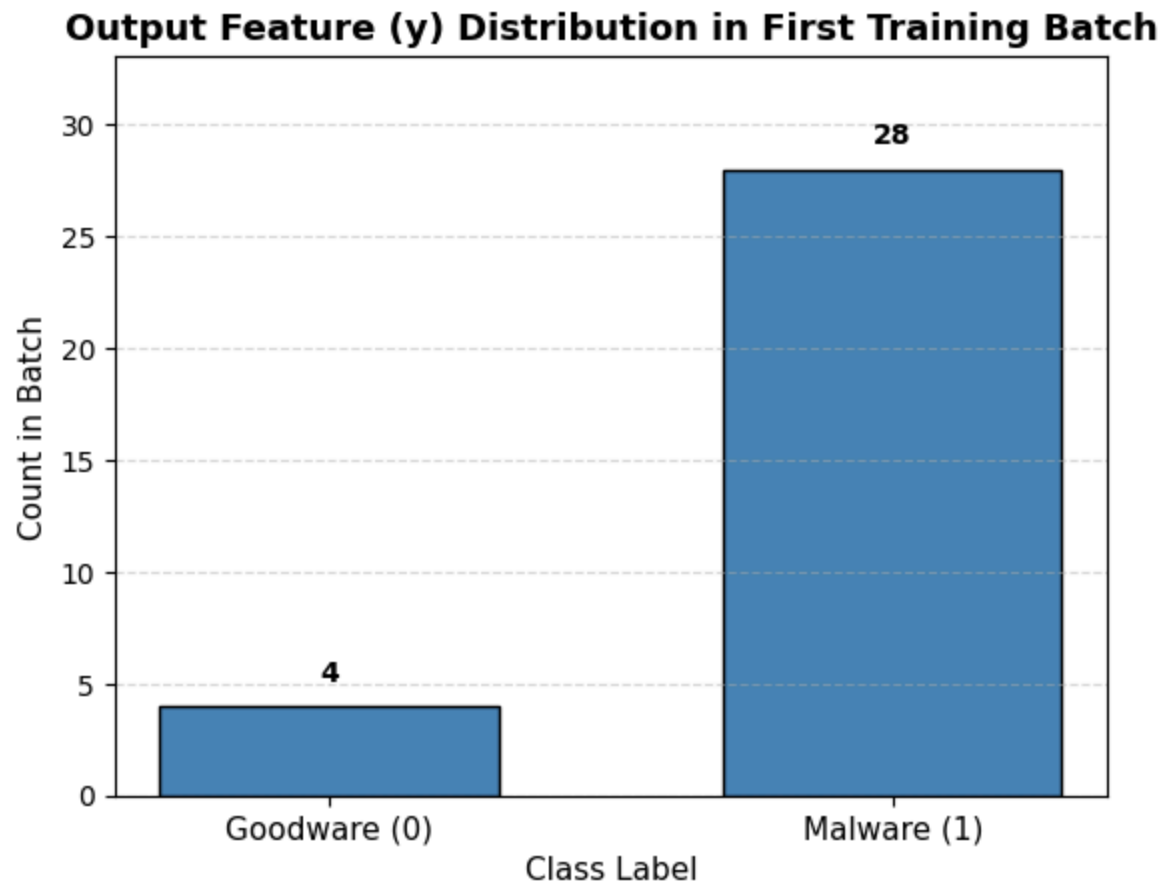
# YOUR CODE STARTS HERE
y_arr = y.detach().cpu().numpy().flatten().astype(int)
# 2. Plot discrete bins centered on 0 and 1
counts, bins, patches = plt.hist(
    y_arr,
    bins=[-0.5, 0.5, 1.5],
    rwidth=0.6,
    color="steelblue",
    edgecolor="black",
)
# 3. Label ticks and add titles/axes matching the earlier EDA style
plt.xticks([0, 1], ["Goodware (0)", "Malware (1)"], fontsize=11)
plt.xlabel("Class Label", fontsize=11)
plt.ylabel("Count in Batch", fontsize=11)
plt.title(

```

```

    "Output Feature (y) Distribution in First Training Batch",
    fontsize=13,
    fontweight="bold",
)
plt.grid(axis="y", linestyle="--", alpha=0.5)
# 4. Annotate count values on top of each bar
for count, patch in zip(counts, patches):
    plt.text(
        patch.get_x() + patch.get_width() / 2,
        count + (max(counts) * 0.03 if max(counts) > 0 else 0.5),
        f"{int(count)}",
        ha="center",
        va="bottom",
        fontweight="bold",
    )
# 5. Set headroom above the tallest bar
plt.ylim(0, max(counts) * 1.18 if max(counts) > 0 else 5)
plt.show()
# YOUR CODE ENDS HERE

```



- **Q4.1):** Say some constant scalar C was added to every logit output o_i . How would the final probabilities be affected? Briefly justify your answer.

- **A4.1):** The probability would not be affected as the constant against all o_i would all cancel out in softmax.
- **Q4.2):** How does exponentiation in the softmax affect the ratio of certain probabilities? Consider an outcome where one logit o_1 is 2, and another logit o_2 is 4. After the softmax, will the ratio of these values be double (as they currently are), less than double, or more than double? Will this strengthen or weaken the confidence of the model on its initial guess?
- **A4.2):** It would be more than double (at least if you are taking $\exp(o_2)/\exp(o_1)$) as exponents would grow exponentially rather than linearly. This make the model more confident and in this example the confidence of o_2 would be $\sim 88\%$ after plugging it into softmax.
- **Q4.3):** Implement `softmax()` in the space below, and briefly describe your code.
- **A4.3):** Originally we used `.sum()` with no `keep dim`, which reduced it to a scalar, the following is revised for part 7. We first shift all values by max to prevent overflow from the exp calculation for part 7, then calculate all exponents with `.exp()`. Then we compute all exp divided by the sum of exponents (we keep dim to ensure it works for multiple outputs)

```
def softmax(0):
    0 = 0 - 0.max(dim=1, keepdim=True).values # Originally we did .sum which
    was fine until part 7 because we need multipl otuput classes
    exp_0 = torch.exp(0) # Calculate all exponents
    return exp_0 / exp_0.sum(dim=1, keepdim=True) # Softmax exp / exp_sum
    # YOUR CODE ENDS HERE
```

```
# Print the softmax function for some arbitrary 2D input
0 = torch.rand((2, 5))
softmax_0 = softmax(0)
print(softmax_0.flatten()) # print whole array
print(softmax_0.max()) # print highest confidence element
print(softmax_0.flatten().sum()) # should return 1
softmax_0, softmax_0.sum(1)
```

```
tensor([0.2405, 0.1518, 0.3031, 0.1422, 0.1623, 0.1732, 0.1655, 0.2683,
        0.2405,
        0.1524])
tensor(0.3031)
tensor(2.)
```

```
(tensor([[0.2405, 0.1518, 0.3031, 0.1422, 0.1623],
        [0.1732, 0.1655, 0.2683, 0.2405, 0.1524]]),
 tensor([1., 1.]))
```

```
class LinearClassifier_v1(d2l.Module):
    """The linear classification model implemented from scratch."""
    def __init__(self, num_inputs, num_outputs=1, lr=0.03, sigma=0.25):
        super().__init__()
        self.save_hyperparameters()

        # YOUR CODE STARTS HERE
        self.W = torch.nn.Parameter(torch.normal(0, sigma, (num_inputs, 1)))
        self.b = torch.nn.Parameter(torch.zeros(1))
        # YOUR CODE ENDS HERE
```

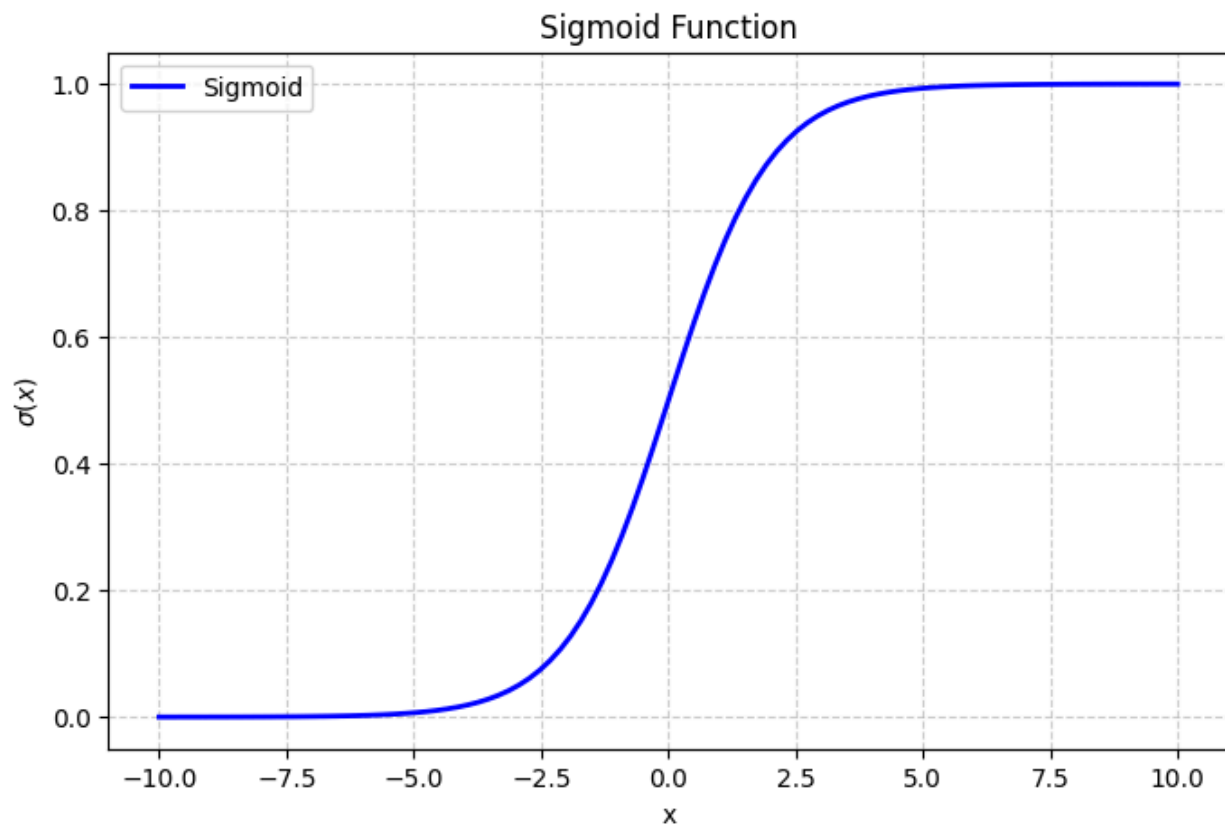
- **Q4.4):** Implement `sigmoid()` in the space below, and visualize the output over a range of input values.
- **A4.4):** Done, the graph looks correct given it has the signature S shape with the middle being at the standard threshold of 0.5

```
def sigmoid(x):
    return 1 / (1 + torch.exp(torch.neg(x)))

# Visualize the sigmoid function on a range of input values from -10 to 10
X = torch.linspace(-10, 10, 100)
sigmoid_X = sigmoid(X)

# YOUR CODE STARTS HERE
# This does looks like the S graph of a sigmoid!
plt.figure(figsize=(8, 5))
plt.plot(X, sigmoid_X, label="Sigmoid", color="blue", linewidth=2)
plt.title("Sigmoid Function")
plt.xlabel("x")
plt.ylabel("$\sigma(x)$")
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()
plt.show()
# YOUR CODE ENDS HERE
```

```
<>:14: SyntaxWarning: invalid escape sequence '\s'
<>:14: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipykernel_40382/2634990771.py:14: SyntaxWarning: invalid escape sequence '\s'
plt.ylabel("$\sigma(x)$")
```



- **Q4.5):** What is the range of our predicted labels $\hat{\mathbf{y}}$? What are the possible values for our actual labels $\mathbf{y}_j$?
- **A4.5):** The range of our predicted values is (0,1) due to sigmoid (softmax in multiclass). The possible values for $\mathbf{y}_j$ is determined based on the amount of classes, but in this case it is either 0, 1.
- **Q4.6):** In the binary classification case (where $q = 1$), given the range of $\hat{\mathbf{y}}$, what possible values can our loss function $l(\mathbf{y}, \hat{\mathbf{y}})$ have? Be specific.
- **A4.6):** It would be either 0 in the case that $\mathbf{y}_1$ is 0 or $\log(\hat{\mathbf{y}})$ if $\mathbf{y}_1$ is 1.
- **Q4.6):** Given the range of $\mathbf{y}$ and $\hat{\mathbf{y}}$, what possible values can our new binary cross-entropy loss function $l(y_i, \hat{y}_i)$ have? What does a "correct" prediction look like? An "incorrect" prediction? Be specific.
- **A4.6):** We kind of have two cases depending on the class of $\mathbf{y}_i$, either $l = -\log(\hat{\mathbf{y}})$ or $-\log(1 - \hat{\mathbf{y}})$. Those are the two possible values which are dependent on the prediction. A correct prediction would have a value closer to 1

inside the log, while a wrong prediction would be closer to 0 inside the log creating a larger loss value.

- **Q4.7):** Implement `loss()` below, and briefly explain your code for this part.
- **A4.7):** It uses torches log function, but otherwise is a 1:1 implementation of the binary cross-entropy above. Except to actually calculate the whole loss we needed to take the mean via `.mean()` which caused issues briefly.

```
def binary_cross_entropy(y_hat, y):  
    return -(y * torch.log(y_hat) + (1 - y) * torch.log(1-y_hat)).mean()  
  
@d2l.add_to_class(LinearClassifier_v1)  
def loss(self, y_hat, y):  
    return binary_cross_entropy(y_hat, y)
```

- **Q4.9):** What line in the code block below converts our predictions from soft probabilities (values in the range [0-1] to hard probabilities (predictions of {0,1})? Briefly explain how that part of the code is working.
- **A4.9):** This takes the predictions as probabilities and the line `preds = (y_hat > 0.5).float()` converts them into hard probabilities as floats (either 0.0 or 1.0) by checking if they are above the threshold 0.5, which converts them to a Boolean and then converting it back to a float. Then it counts the number of correct below with `y_true`.
- **Q5.2):** How did the performance of the concise model compare to our from-scratch model? What was the highest train and validation accuracy percentage for each?
- **A5.2):** Its about the same, I would say within margin of error from entropy (random data sampling). Our training accuracy was 96% and our validation was also just about 96%. This is within 1% difference of the original model

```
# Create a MalwareDataset object using the original dataset  
malware_df= pd.read_csv("data/TUANDROMD.csv")  
MalwareData_v2 = MalwareDataset(malware_df)  
  
# Generate first iteration of a dataloader  
X, y = next(iter(MalwareData.train_dataloader()))  
num_inputs = X.shape[1]  
MalwareModel_v2 = LinearClassifier_v2(lr=0.03)
```

```

# Instantiate the trainer, choosing a number of iterations (epochs)
# Then, fit the trainer with the data
model_trainer_v2 = d2l.Trainer(max_epochs = 10)

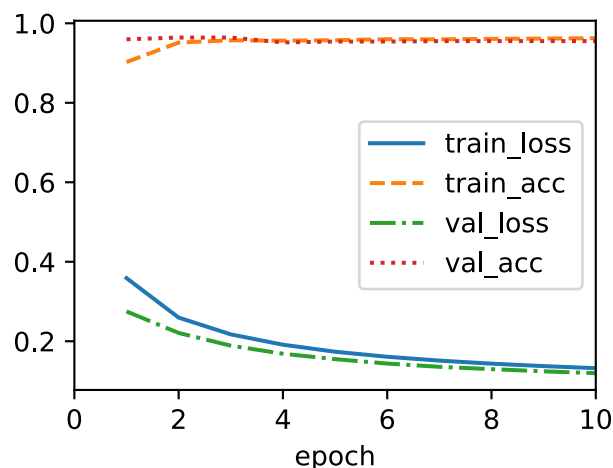
# This manually instantiates the matplotlib board and forces the flag to True
model_trainer_v2.plot_flag = True
#model_trainer_v2.board = d2l.ProgressBar(xlabel='batches')
MalwareModel_v2.board.xlabel= 'epoch'
model_trainer_v2.fit(MalwareModel_v2, MalwareData)

# Function to print out final accuracies and loss values. We will be doing
this fairly frequently, so save as a helper function
def print_model_results(board):
    # Print out final accuracies and loss values:
    final_tr_loss = float(board.data['train_loss'][-1][1])
    final_tr_acc = float(board.data['train_acc'][-1][1])
    final_val_loss = float(board.data['val_loss'][-1][1])
    final_val_acc = float(board.data['val_acc'][-1][1])

    print("\n" + "="*40)
    print(f"Final Training Loss:      {final_tr_loss:.4f}")
    print(f"Final Validation Loss:      {final_val_loss:.4f}")
    print("-"*40)
    print(f"Final Training Accuracy:      {final_tr_acc * 100:.2f}%")
    print(f"Final Validation Accuracy: {final_val_acc * 100:.2f}%")
    print("="*40)

print_model_results(model_trainer_v2.model.board)

```



```

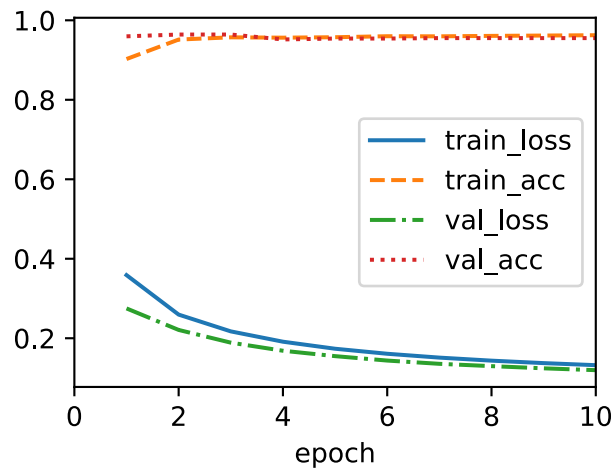
=====
Final Training Loss:      0.1322
Final Validation Loss:    0.1197
-----

```

```

Final Training Accuracy:   96.25%
Final Validation Accuracy: 95.52%
=====

```



- **Q6.1:** What is the most frequent image number in the first 100 images in the MNIST training dataset? The least frequent? Which pixel locations in the first 100 images have the most intensity on average? The least intensity? Use the following code block to explore answers to these questions. This MNIST dataset is designed so that you can use `.targets` to access the numerical labels (0-9), and `.data` to access the images themselves. These are tensors that you can grab and slice appropriately to answer these questions.

- **A6.1:**

```
# Find the most frequent label in the first 100 training images DONE & least
# Find the average intensity of the pixel at each position across the first
100 training images

# Plot a histogram of the output feature in the first training batch. Show all
10 classes in the histogram, and make sure to label the axes and title of the
plot.

# Next, create a new figure, and calculate the average intensity of the pixel
at each position across the first 100 training images and plot it as an image
DONE

# YOUR CODE STARTS HERE
labels_100 = dataset.targets[0:100]
data_100 = dataset.data[0:100]
counts = torch.bincount(labels_100)
most_freq = torch.argmax(counts).item()
least_freq = torch.argmin(counts).item()

print(f>Data_100 shape: {data_100.shape}")
```

```

avg_intensity = np.average(data_100.numpy(), axis=0) # Find average intensity

print(f"Most frequent in first 100: {most_freq}")
print(f"{most_freq} appears {counts[most_freq].item()} times")
print("=====")
print(f"Least Frequent in first 100 {least_freq}")
print(f"{least_freq} appears {counts[least_freq].item()} times")

#print(f"Intensity shape: {avg_intensity}") # This outputs what appears to be
slop... Dont uncomment

# Print histogram
plt.hist(labels_100, bins=10, edgecolor="purple", color="gold", alpha=0.67)
plt.title("Counts for each label in first 100")
plt.show()

# Print average intensity
plt.imshow(avg_intensity, cmap='viridis')
plt.colorbar()
plt.title("Average intensity accross first 100 images")
plt.show()

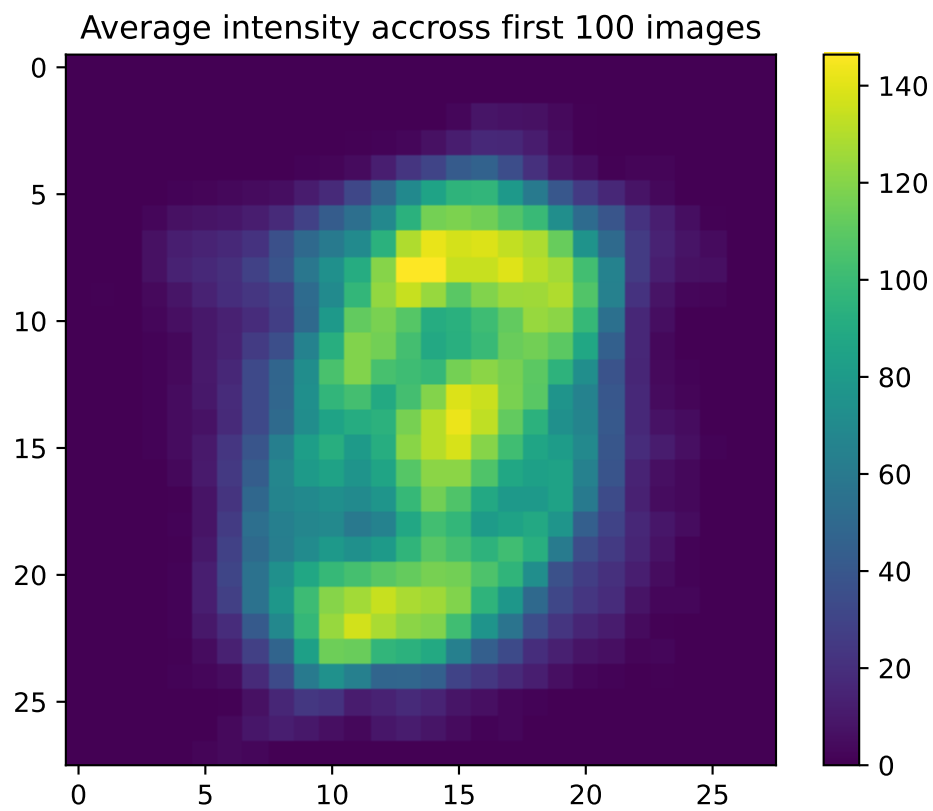
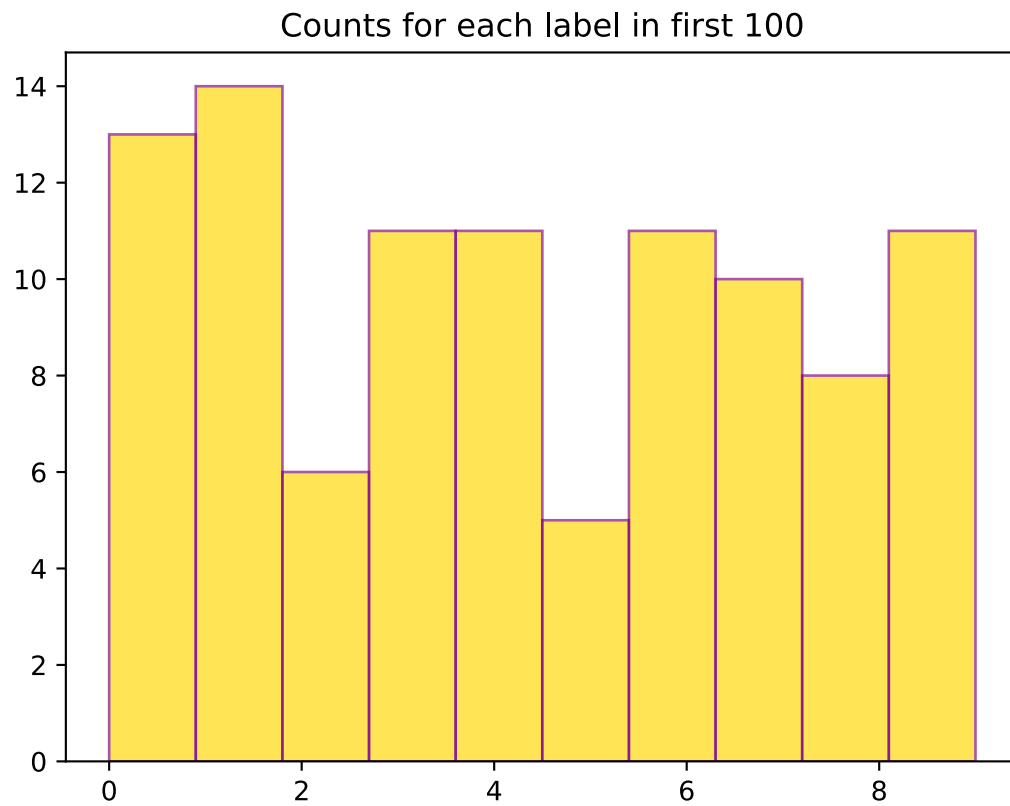
# YOUR CODE ENDS HERE

```

```

Data_100 shape: torch.Size([100, 28, 28])
Most frequent in first 100: 1
1 appears 14 times
=====
Least Frequent in first 100 5
5 appears 5 times

```



- **Q6.2:** Implement the normalization of our image set as a function called `normalize_images()` that takes as input the images from our

`dataset.data[]` , calculates the mean and standard deviation (across all images), and transforms the images so that the new mean is 0, and standard deviation is 1. Briefly explain your code below.

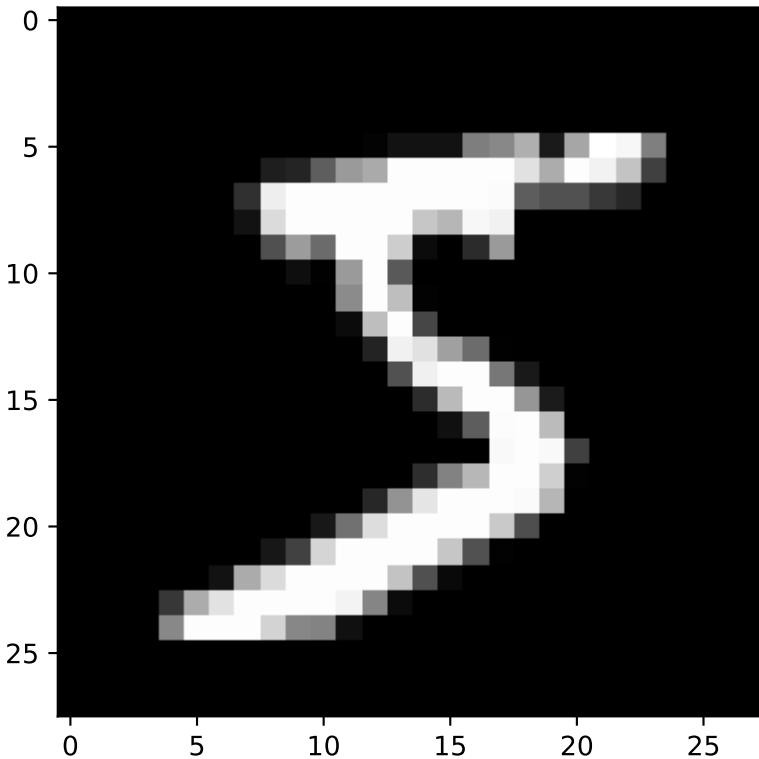
- **A6.2:** First we calculate the average as the average across all pixels, then the standard deviation across all pixels. Then we get a "shifted" value which moves the average to (near, due to the nature of fp) 0 and then divide by the standard deviation. WE verified this with the print statements but commented them out. We also got confused as we were getting a 5 so we verified that the label is 5 and the labels and images didnt shift.

```
# Normalize the pixel intensity values to have a mean of 0, and a standard deviation of 1 across the entire training dataset. Then, plot the average pixel intensity for the normalized images that have a label of "3" again as an image.
def normalize_images(image_dataset):
    average = torch.mean(image_dataset.float())
    standard = torch.std(image_dataset.float()) #maybe an edge case if this is 0???
    # print(f"Avg: {average}")
    # print(f"Standard: {standard}")
    shifted = image_dataset - average
    # print(f"Shifted avg: {torch.mean(shifted)}") #close enough to 0
    normed = shifted / standard
    # print(f"Norm shape: {normed.shape}")
    # print(f"Norm avg: {torch.mean(normed)}")
    return normed

normalized_data = normalize_images(dataset.data)
plt.figure()
plt.title(f"Figure: {dataset.targets[0]}") #sanity check as the output was 5???
plt.imshow(normalized_data[0], cmap='gray') # Figure should look the same as before, just with different pixel intensity values
```

<matplotlib.image.AxesImage at 0x7f92dc3492e0>

Figure: 5



- **Q6.3:** Review the implementation of `MNISTDataset` below, and briefly answer the following questions. Where in the code do we flatten our 28x28 image to a 768x1 vector? What is our `torch.empty_like` function doing?
- **A6.3:** it is the `self.X = normalized_data.view(-1, 28*28)` section which flattens it down into 784x1 (technically 100000x784 but that's because of the number of images). `torch.empty_like` is not used (this is a typo in the question).

```
mnist_train = torchvision.datasets.MNIST(root='./data', train=True,
download=True)
mnist_val = torchvision.datasets.MNIST(root='./data', train=False,
download=True)

MNIST_Data = MNISTDataset(mnist_train, mnist_val, batch_size=32)

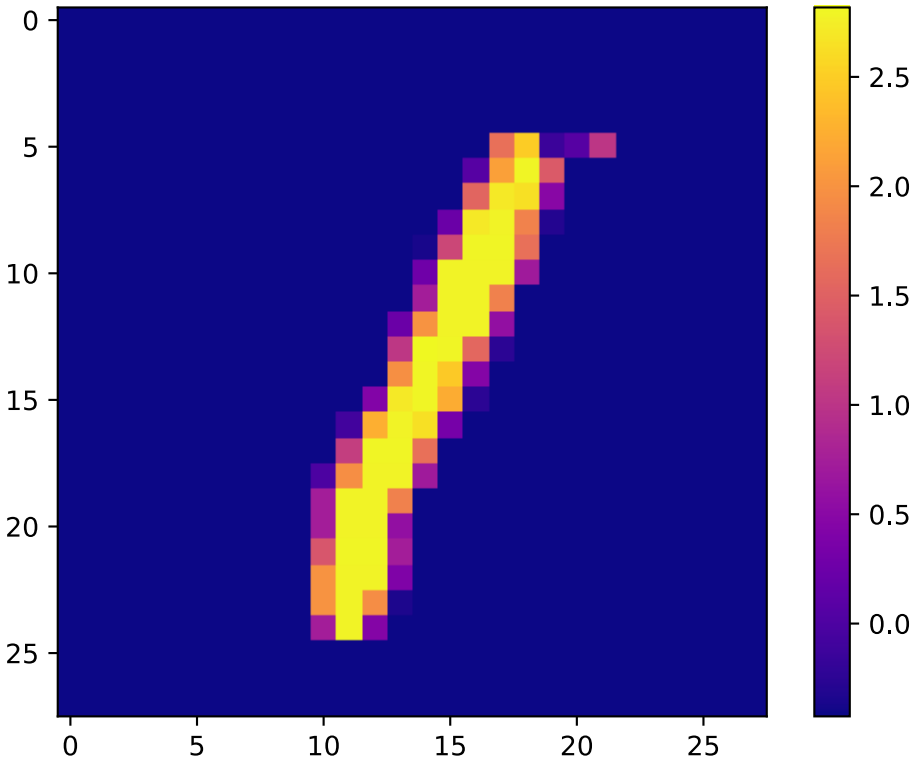
# Review how the first batch looks
X, y = next(iter(MNIST_Data.train_dataloader()))
print(X.shape, y.shape)

# YOUR CODE STARTS HERE
plt.imshow(X[15].reshape(28,28), cmap="plasma")
plt.title(f"This is a {torch.argmax(y[15])}, pretty cool right?")
plt.colorbar()
```

```
plt.show()
# YOUR CODE ENDS HERE

torch.Size([70000, 784])
torch.Size([70000, 10])
60000 10000
torch.Size([32, 784]) torch.Size([32, 10])
```

This is a 1, pretty cool right?



- **Q7.1)** Implement the `__init__` constructor for class `LinearRegression_v3` below, and describe what differences (if any) you made from the previous `LinearRegression_v1.__init__()`:
- **A7.1)** We had to change from a hardcoded 1 for `num_outputs` to `num_outputs` to output the 10 classes

```
class LinearClassifier_v3(d2l.Module):
    def __init__(self, num_inputs, num_outputs=1, lr=0.03, sigma=0.25):
        super().__init__()
        self.save_hyperparameters()

        # YOUR CODE STARTS HERE
        self.W = torch.nn.Parameter(torch.normal(0, sigma, (num_inputs,
num_outputs)))
        self.b = torch.nn.Parameter(torch.zeros(num_outputs))

        # Keep the same SGD optimization
```

```
def configure_optimizers(self):
    return torch.optim.SGD(self.parameters(), self.lr)
```

```
@d2l.add_to_class(LinearClassifier_v3) #@save
def forward(self, X):
    return softmax(torch.matmul(X, self.W) + self.b)
```

- **Q7.2)** This forward pass was implemented with completely randomized weights. Review the predicted outputs - for the 32 images that are classified in this batch, what labels (numbers) would they be predicted to be? Briefly defend your answer.
- **A7.2)** They would likely be randomly predicted, but the prediction itself would be the max of the probability of the outputs (i.e. the largest label prediction). This is because all the values sum to 1.0 and as such there must exist a prediction that is the largest which is chosen as the final output label.
- **Q7.3)** Implement function `cross_entropy()` and `loss()` below. Note that the "correct" (or at least most Pythonic, and likely most computationally efficient) approach would be use a [list comprehension](#) here. For whatever approach you do use, explain your implementation and how the terms of the cross-entropy equation are being computed.
- **A7.3)** Just like the equation above, it compares the negative log of the prediction for the real class label and sums it for the whole batch.

```
def cross_entropy(y_hat, y):
    return -(y * torch.log(y_hat)).sum(dim=1).mean()

@d2l.add_to_class(LinearClassifier_v3)
def loss(self, y_hat, y):
    return cross_entropy(y_hat, y)
```

- **Q7.4)** For a randomized (arbitrary) prediction of an image's class, what would you expect the typical range of the `cross_entropy()` loss function to be? Either use the code block below to explore the answer to this question, or provide a more detailed arithmetic explanation.
- **A7.4)** A randomized prediction of `cross_entropy()` we would expect it to be - $\ln(0.1)$ or 2.3 but we are getting ~11-12. This seemed very far off, however, the way we are initializing weights to be randomized by a large margin we are

getting confidently wrong predictions rather than a baseline 0.1 across the board, hence the higher loss.

```
# Test out the loss function with our initial prediction and labels
# YOUR CODE STARTS HERE
loss_1 = MNIST_Model.loss(y_hat=y_hat1, y=y)
print(f"Loss: {loss_1}")
# YOUR CODE ENDS HERE
```

Loss: 10.219476699829102

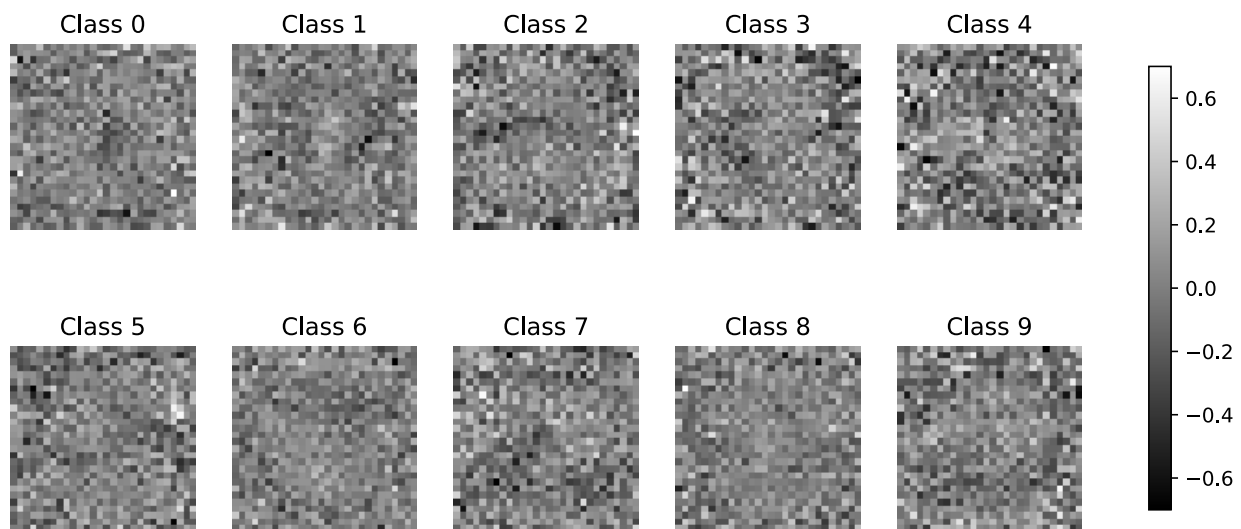
- **Q7.5)** Implement code in the following block that reads back the trained weights from our `MNIST_model` object, and visualizes the weights corresponding to detecting class labels as a 2 row by 5 column grid. If you are having trouble doing this for all classes, at the very least visualize the weights corresponding to label `7`. Describe your observations: do the values of these weights make some intuitive sense to you?
- **A7.5)** It kind of does, its very rough but you can see some shapes forming in the classes through the noise, particularly class 2 and 3. The lack of patterns does make me think that maybe we messed up somewhere as it doesn't look as good as it might otherwise.

```
# Read the weights back from the trained model, and reshape them into a 28x28
image for each output class.
# Then, plot these weight images in a 2 row by 5 column grid, and set the
title of each image to its corresponding class.

# YOUR CODE STARTS HERE
W = MNIST_model.W.detach().cpu()
fig, axes = plt.subplots(2, 5, figsize=(12, 5))

for c, ax in enumerate(axes.flat):
    img = W[:, c].reshape(28, 28)
    im = ax.imshow(img, cmap='grey', vmin=-img.abs().max(),
vmax=img.abs().max())
    ax.set_title(f"Class {c}")
    ax.axis('off')

fig.colorbar(im, ax=axes.ravel().tolist(), shrink=0.8)
plt.show()
# YOUR CODE ENDS HERE
```



- **Q8.1)** The numbers `0.1307` and `0.3081` for the mean and standard deviation seem a little arbitrary. Reviewing our prior pre-processing of the MNIST dataset, how were these values derived? Be specific.
- **A8.1)** They appear to be calculated as the average pixel intensity and the standard deviation for the normalized dataset. Rather than being 33.33 and ~ 78 they are normalized to be between 0,1.
- **Q8.2)** In the code block below, grab a batch of images from the MNIST validation set. Evaluate your model on these images, and plot one image that is correctly predicted, and one that is incorrectly predicted. What is your best guess as to why the one image was predicted incorrectly?
- **A8.2)** The correct one is a very clear 7, so that makes sense. The first incorrect is the worst drawn 6 I have ever seen and it got it wrong as a 5, which honestly it does look vaguely like a 5 and not a 6 so It is understandable on why it would get that wrong.

```
# Grab the next batch of validation data

# Evaluate the model on the data, and find the first correct and the first
incorrect prediction

# Plot the correctly predicted image with label

# Plot the incorrectly predicted image with label

# YOUR CODE STARTS HERE
```

```

# Visualize the weights again, just because I would like to see if its
different
# Yes they look much better
W = MNIST_model2.net[1].weight.detach().cpu() # (10, 784)
fig, axes = plt.subplots(2, 5, figsize=(12, 5))

for c, ax in enumerate(axes.flat):
    img = W[c].reshape(28, 28)
    im = ax.imshow(img, cmap='gray', vmin=-img.abs().max(),
vmax=img.abs().max())
    ax.set_title(f"Class {c}")
    ax.axis('off')

fig.colorbar(im, ax=axes.ravel().tolist(), shrink=0.8)
plt.show()

# Actual evalute for first correct and wrong
X, y = next(iter(MNIST_Data2.val_dataloader()))

MNIST_model2.eval()
with torch.no_grad():
    preds = MNIST_model2(X).argmax(dim=1)

correct_idx = (preds == y).nonzero()[0].item()
wrong = (preds != y).nonzero()
if len(wrong) == 0:
    print("none wrong, maybe our model is perfect?")
wrong_idx = wrong[0].item()

fig, axes = plt.subplots(1, 2, figsize=(7, 3.5))

# Plot the correctly predicted image with label
axes[0].imshow(X[correct_idx].squeeze(), cmap='gray')
axes[0].set_title(f"Correct\nLabel: {y[correct_idx].item()} | Pred:
{preds[correct_idx].item()}")
axes[0].axis('off')

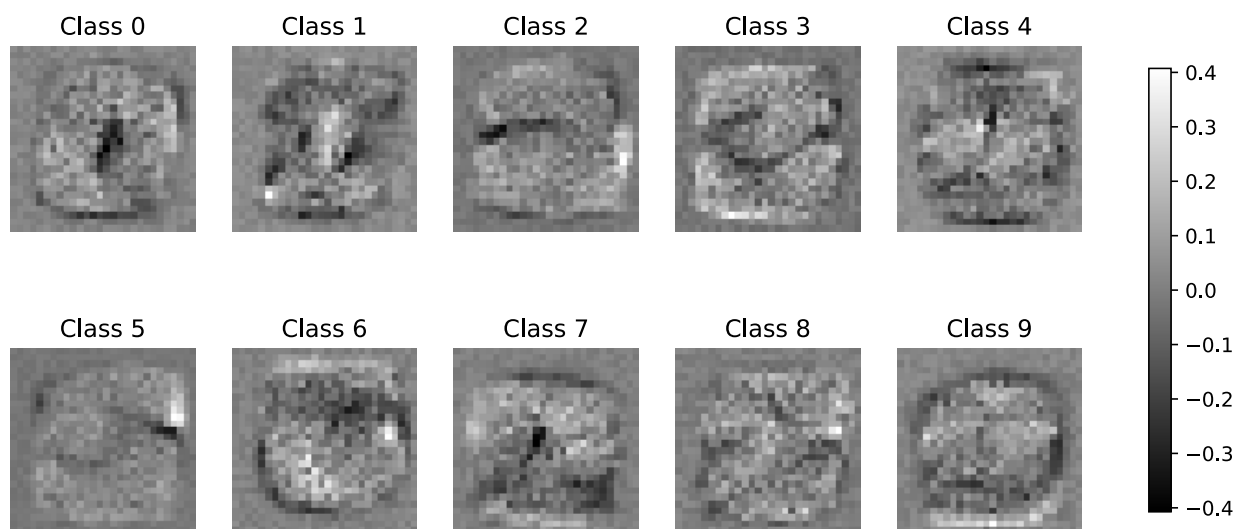
# Plot the incorrectly predicted image with label
axes[1].imshow(X[wrong_idx].squeeze(), cmap='gray')
axes[1].set_title(f"Incorrect\nLabel: {y[wrong_idx].item()} | Pred:
{preds[wrong_idx].item()}")
axes[1].axis('off')

plt.tight_layout()
plt.show()

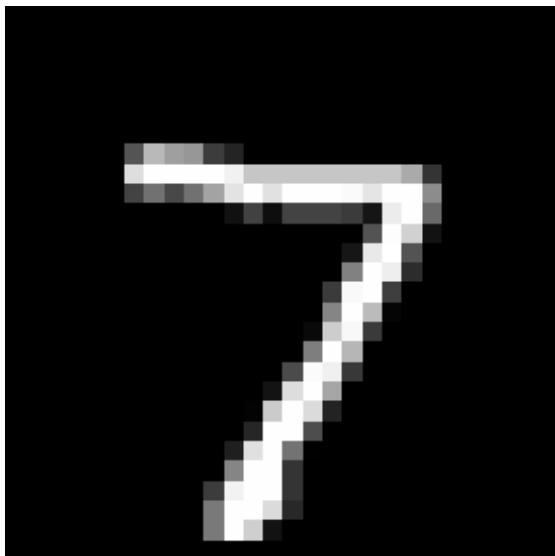
print(f"Batch accuracy: {(preds == y).float().mean().item() * 100:.2f}%")

# YOUR CODE ENDS HERE

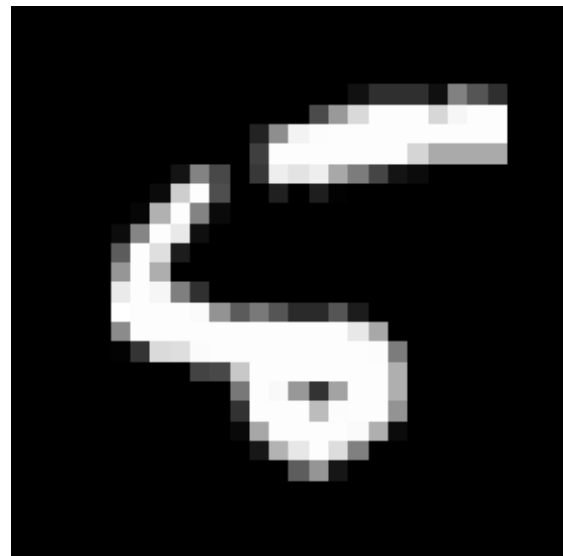
```



Correct
Label: 7 | Pred: 7



Incorrect
Label: 5 | Pred: 6



Batch accuracy: 93.75%

Prelab Questions

- **Q0.1):** A linear classifier can be applied to both tabular malware data and image data such as MNIST. In what ways are these problems similar, and in what ways are they different? Consider features, interpretability, dimensionality, and model assumptions.
- **A0.1):** These problems are similar as they are both classification, however in the malware dataset the feature inputs are much better defined compared to the raw image inputs. The image inputs are much more difficult to properly do feature extraction beyond basic pre processing, however the MNIST dataset is still relatively basic in this regard. In comparison the malware dataset where the

features are already extracted. The largest difference is that the activation function will be different as the MNIST dataset and Malware dataset are multi-class and binary classification respectively.

In terms of dimensionality the malware has a smaller dimensionality, while the MNIST dataset gets flattened into $W \times H$ dims.

- **Q0.2):** How is binary classification different from multi-class classification? Consider the model output, modeling process, and challenges involved.
- **A0.2):** The main difference is instead of predicting between two classes (-1, 1 or 0, 1) you are predicting between multiple classes. This is achieved through a different activation function, softmax for multi and sigmoid for binary. There is also additionally pre-processing of the data for each as you need to turn each label into a numeric value.
- **Q0.3):** What kinds of problems might a linear classifier be effective with, or alternately, struggle with? Be specific.
- **A0.3):** A linear classifier would be effective with data that is easily separable but would struggle with complex non-linear data or perceptual data with complex features like images, audio, video.